Lab 6 Build Guide

Lab 6 Build Guide: Drift Detection and Reconciliation

This guide walks through detecting when the network's actual state diverges from the declared intent, and three strategies for bringing them back in sync. By the end you will have a drift detection engine that catches out-of-band changes, reconciliation tools for different scenarios, and a brownfield import tool for existing networks.

What We Built

Labs 1 through 5 established a complete pipeline: define intent, validate it, generate configs, deploy them, and verify the result. That pipeline assumes the only path to the network is through the automation. But in the real world, people SSH into routers. Someone fixes an emergency at 3 AM by typing commands directly. Someone tests a change "just to see what happens" and forgets to revert it. These out-of-band changes are drift.

Drift is the gap between what the data model says the network should look like and what the network actually looks like. This lab builds three things to deal with it.

The drift detection engine connects to every device, pulls the running config, and compares it against the intended config generated from the data model. Any difference is flagged with the specific section and line that changed.

The reconciliation module offers three strategies depending on the situation: push the intended config back (auto-remediate), generate a report for human review, or accept the running config as the new truth (absorb).

The brownfield import tool handles the case where you are adopting Network as Code on an existing network. It pulls configs from all devices and generates a starting-point data model from what is actually running.

Prerequisites

You need Labs 1-5 complete with the fabric deployed and all post-change tests passing.

```
uv run python -m scripts.fabric_status
uv run pytest tests/post_change/ -v
```

All 6 devices UP, 31 post-change tests passing.

Part 1: Drift Detection

The drift detection engine lives at `drift/detect.py`.

```
cat drift/detect.py
```

How It Works

The engine generates the intended configs from the data model (the same configs the deploy script pushes), then pulls the running config from each device via `docker exec`. It normalizes both configs to strip cosmetic differences (whitespace, comments, FRR-internal reformatting) and compares them line by line.

The normalizer is the critical piece. FRR reformats configs internally: it converts `network` statements to interface-level `ip ospf area`, converts `passive-interface lo` to `ip ospf passive`, and reorders some commands. These are not drift. The normalizer strips all of these so the comparison only catches meaningful differences.

When drift is found, the engine identifies which config section is affected (hostname, interfaces, OSPF, or BGP) and what specifically changed. This gives operators a targeted view instead of a wall of diff output.

Run It on a Clean Fabric

```
uv run python -m drift.detect
```

You should see all 6 devices showing CLEAN. This confirms that the deployed configs match the intent.

Introduce Drift

Now simulate an out-of-band change. Connect to spine1 and remove the BGP cluster-id:

```
docker exec clab-nac-spine-leaf-spine1 vtysh -c "conf t" -c
        "router bgp 65000" -c "no bgp cluster-id" -c "end"
```

Run drift detection again:

```
uv run python -m drift.detect
```

Spine1 now shows DRIFT with one item: `bgp cluster-id 10.0.0.1` is in the intended config but missing from the running config. The other 5 devices are still CLEAN.

Single Device Check

To check only one device:

```
uv run python -m drift.detect --device spine1
```

JSON Output

For programmatic consumption (CI pipelines, monitoring tools):

```
uv run python -m drift.detect --json
```

Part 2: Reconciliation

The reconciliation module lives at `drift/reconcile.py` with three subcommands.

```
cat drift/reconcile.py
```

Strategy 1: Auto-Remediate

This pushes the intended config back to the drifted device. Use it when the drift is clearly wrong and should be corrected immediately.

With the cluster-id still removed from spine1:

```
uv run python -m drift.reconcile remediate --device spine1
```

The script detects the drift, pushes the intended config via vtysh, and reports success. Verify it worked:

```
uv run python -m drift.detect --device spine1
```

Back to CLEAN.

Strategy 2: Report for Review

This generates a drift report without changing anything on the network. Use it when you want a human to review the drift before deciding what to do.

Introduce drift again on spine1 (same command as before), then:

```
uv run python -m drift.reconcile report
```

The report is saved to `reports/` with a timestamp and also printed to the console. It shows each drifted device, the number of drift items, and the specific intended vs actual values.

Strategy 3: Absorb

This pulls the running config from a device and saves it as the new intended config. Use it when someone made an emergency change that should become the new baseline.

```
uv run python -m drift.reconcile absorb --device spine1
```

The script backs up the old intended config to `spine1.conf.bak`, pulls the running config, and saves it as the new `spine1.conf`. Now drift detection will show CLEAN for spine1 because the intended config matches the running config.

The absorb strategy only updates the generated config file. It does not update the YAML data model. To make the change permanent, you would need to update the data model manually to reflect the new intent and regenerate all configs.

After the demo, restore the original config:

```
uv run python -m generators.python.render
uv run python -m deploy.scrapli_deploy
```

Part 3: Brownfield Import

The brownfield import tool lives at `drift/brownfield_import.py`.

```
cat drift/brownfield_import.py
```

What It Does

The tool connects to every running ContainerLab container, pulls the running config, parses it using regex patterns, and generates YAML files that represent the current state of the network. It extracts:

- Device hostnames and roles (spine if it has a cluster-id, leaf otherwise)
- Loopback IP addresses
- BGP ASN and neighbor lists
- OSPF router-ids
- Interface IPs and descriptions

Run It

```
uv run python -m drift.brownfield_import
```

The output goes to `data-imported/` by default. Check what it generated:

```
ls data-imported/
cat data-imported/topology.yaml
cat data-imported/fabric.yaml
```

The topology file has all 6 devices with their roles, loopbacks, and ASNs. The fabric file has the shared ASN. The overlay file lists the route reflectors.

Why It Matters

Most networks are not greenfield. If you want to adopt Network as Code on an existing fabric, you need a way to get the current state into a data model. The brownfield import gives you a starting point. You clean up the YAML, add the fields the parser could not extract (like VRFs and network segments), and you have a data model that represents your real network.

Part 4: Scheduled Drift Checks

The GitHub Actions workflow at `.github/workflows/drift_check.yaml` runs drift detection on a schedule.

```
cat .github/workflows/drift_check.yaml
```

It runs every 6 hours (configurable via the cron expression) and can also be triggered manually from the GitHub Actions tab. If drift is detected, it automatically creates a GitHub issue with the drift report so the team is notified.

The workflow requires a self-hosted runner with access to the ContainerLab management network, same as the deploy workflow from Lab 4.

Part 5: Commit the Drift Detection Module

```
git status
```

You should see the new `drift/` directory with 4 files and the drift check workflow.

```
git add drift/ .github/workflows/drift_check.yaml docs/lab6-build-
        guide.md
git commit -m "Lab 6: drift detection, reconciliation, and
        brownfield import"
```

Check the log:

```
git log --oneline
```

Six commits. The automation stack now covers the full lifecycle including drift management. The data model is not just the initial state but the continuously enforced truth.

Part 6: What We Proved

By the end of this lab you have demonstrated three things.

First, that drift detection is practical and precise. The engine catches a single removed line on one device out of six, identifies the affected section, and reports the specific change. It does not flood you with false positives from cosmetic FRR reformatting.

Second, that reconciliation is not one-size-fits-all. Auto-remediation is fast but aggressive. Reporting gives humans the final say. Absorbing acknowledges that sometimes the network knows better than the data model. Having all three strategies means you can match the response to the situation.

Third, that brownfield adoption is possible. You do not need to start from zero. The import tool gives you a starting point by reading what is actually on the network and translating it into the data model format. The output needs cleanup, but it is dramatically faster than building a data model by hand for an existing network.

Troubleshooting

Drift detection shows false positives: The normalizer filters cosmetic differences like FRR reformatting, comments, and whitespace. If you see drift items that are not real changes, add the pattern to the `skip_patterns` tuple in `_normalize_config()`.

Remediation says "config pushed with warnings": Some vtysh warnings are benign (like the `vtysh.conf` not found message). Check the running config after remediation to confirm the intended state was restored.

Absorb does not update the data model: By design. Absorb only updates the generated config file. Updating the YAML data model from a running config would require reverse-engineering the intent, which is the brownfield import tool's job.

Brownfield import shows "leaf" for border devices: The parser determines role based on whether the device has a BGP cluster-id (spine) or not (everything else is "leaf"). Border leaf detection would require additional heuristics like checking for external BGP peers or VRF count. The output is a starting point that needs manual refinement.

Scheduled drift check not running: The workflow uses `runs-on: self-hosted` and requires a registered runner. It also needs the `workflow_dispatch` trigger enabled for manual runs from the Actions tab.